

# Les structures de controle élémentaires en Python



## Les Conditions (Structures Conditionnelles)

Les conditions permettent d'exécuter un code **uniquement si une expression est vraie**.

### Structure if (Si)

Exécute un bloc de code si la condition est vraie.

```
root@python:~$
nombre = 10
if nombre > 5:
    print("Le nombre est supérieur à 5.")
# Affiche : Le nombre est supérieur à 5.
```

**Explication :** L'instruction print est exécutée car la condition nombre > 5 est vraie.

### Structure if...else (Si...Sinon)

Exécute un premier bloc de code si la condition est vraie, **sinon** exécute un second bloc de code.

```
root@python:~$
age = 15
if age ≥ 18:
    print("Vous êtes majeur.")
else:
    print("Vous êtes mineur.")
# Affiche : Vous êtes mineur.
```

**Explication :** La condition age ≥ 18 est fausse, donc le code sous le else est exécuté.

### Structure if...elif...else (Si...Sinon Si...Sinon)

Permet de tester **plusieurs conditions** en séquence.

```
root@python:~$
note = 12
if note ≥ 15:
    print("Très bien")
elif note ≥ 10:
    print("Bien")
else:
    print("Insuffisant")
# Affiche : Bien
```

**Explication :** La première condition (≥ 15) est fausse. La deuxième (elif note > 10) est vraie, donc le code sous le elif est exécuté et le reste est ignoré.



## Les Boucles (Itérations)

Les boucles permettent de répéter un bloc d'instructions. C'est essentiel pour automatiser des tâches ou parcourir des collections de données.

### La Boucle for (Itération Définie)

Elle est utilisée lorsque l'on connaît le nombre d'itérations à l'avance, souvent pour parcourir les éléments d'une séquence (comme une liste) ou un intervalle de nombres grâce à la fonction range().

```
root@python:~$
# Utilisation de range(n) pour répéter n fois (de 0 à n-1)
for i in range(3):
    print(f"Tour numéro {i}") # print permet d'afficher
# Affiche :
# Tour numéro 0
# Tour numéro 1
# Tour numéro 2
```

**Explication :** range(3) génère la séquence 0, 1, 2. La boucle s'exécute 3 fois.

### La Boucle while (Itération Conditionnelle)

Elle s'exécute **tant qu'une condition reste vraie**. Il faut s'assurer que la condition devienne fausse à un moment pour éviter une **boucle infinie**.

```
root@python:~$
compteur = 0
while compteur < 3:
    print(f"Compteur : {compteur}")
    # Incrémentation pour éviter la boucle infinie
    compteur = compteur + 1
# Affiche :
# Compteur : 0
# Compteur : 1
# Compteur : 2
```

**Explication :** Le code dans le while s'exécute tant que la variable compteur est strictement inférieure à 3. À chaque tour, on augmente compteur de 1, ce qui permet à la boucle de se terminer.



## Opérateurs Logiques (not, and, or)

Ils permettent de combiner ou d'inverser des conditions pour des tests plus complexes.

| Opérateur | Signification | Quand est-ce Vrai ?                                             |
|-----------|---------------|-----------------------------------------------------------------|
| and       | ET logique    | Si toutes les conditions sont Vraies.                           |
| or        | OU logique    | Si au moins une des conditions est Vraie.                       |
| not       | NON logique   | Inverse la condition (Vrai devient Faux, et Faux devient Vrai). |

```
root@python:~$
# Exemple avec AND
temperature = 25
enseleille = True
if temperature > 20 and enseleille:
    print("Journée idéale pour une sortie.")
# Affiche : Journée idéale pour une sortie.

# Exemple avec OR
heure = 8
jour = "dimanche"
if heure < 9 or jour == "dimanche":
    print("C'est encore l'heure de dormir.")
# Affiche : C'est encore l'heure de dormir.

# Exemple avec NOT
est_finit = False
if not est_finit:
    print("Le travail continue.")
# Affiche : Le travail continue.
```

**Explication :**



- La condition est vraie car `temperature > 20` (`25 > 20`) est VRAI ET ensoleille est VRAI.

- La condition est vraie car `heure < 9` (`8 < 9`) est VRAI. Même si `jour == "dimanche"` est Faux, le OR rend l'ensemble VRAI.
- La condition est VRAIE car `not False` est VRAI. On exécute le bloc de code si la variable est Fausse.

FOLLOW  
THE  
PYTHONIC  
WAY

HACK  
AGAINST  
MACHISM